

Fakultet tehničkih nauka
Univerzitet u Novom Sadu

Paralelne i distribuirane arhitekture i jezici

Osobine, životni vekovi, rukovanje greškama

Branislav Ristić

Zimski semestar, školska 2026./27.

Osobine

Osobine (*Traits*)

- Osobina (*trait*) definiše zajedničko ponašanje kroz metode.
- Različiti tipovi mogu da implementiraju istu osobinu: polimorfizam bez nasleđivanja.

Osobine: deklaracija i implementacija

- Osobina se deklariše ključnom rečju `trait`, a u njoj se navode potpisi metoda.
- Implementacija za konkretan tip piše se kao `impl Osobina for Tip` i definiše te metode.

Osobine: primer

primeri/traits-lifetimes-errors/src/bin/traits.rs

```rust
// Summary can be implemented on NewsArticle and Tweet because
// the trait or the type is local to our crate.
// External traits can't be implemented on external types (e.g. Display
// on Vec<T>): the orphan rule prevents conflicts across crates.

pub trait Summary {
    fn summarize(&self) -> String;
}

pub struct NewsArticle {
    pub headline: String,
    pub location: String,
    pub author: String,
    pub content: String,
}

// Implement Summary on NewsArticle since it's local to this crate
impl Summary for NewsArticle {
    fn summarize(&self) -> String {
        format!("{}, by {} ({})", self.headline, self.author,
        ↪  self.location)
    }
}

pub struct Tweet {
    pub username: String,
    pub content: String,
    pub reply: bool,
    pub retweet: bool,
}

// Implement Summary on Tweet, another local type
impl Summary for Tweet {
    fn summarize(&self) -> String {
        format!("{}: {}", self.username, self.content)
    }
}

fn main() {
    let tweet = Tweet {
        username: String::from("horse_ebooks"),
```

```
        content: String::from("of course, as you probably already know,
 ↪  people"),
        reply: false,
        retweet: false,
    };
    let news_article = NewsArticle {
        headline: String::from("Penguins win the Stanley Cup
 ↪  Championship!"),
        location: String::from("Pittsburgh, PA, USA"),
        author: String::from("Iceburgh"),
        content: String::from(
            "The Pittsburgh Penguins once again are the best hockey team
             ↪  in the NHL.",
        ),
    };

    println!("1 new tweet: {}", tweet.summarize());
    println!("New article: {}", news_article.summarize());
}
```

Izlaz

```
1 new tweet: horse_ebooks: of course, as you probably already know, people
New article: Penguins win the Stanley Cup Championship!, by Iceburgh
 ↪  (Pittsburgh, PA, USA)
```

Osobine: pravilo siročeta (*orphan rule*)

- Implementacija je dozvoljena samo ako je osobina ili bar jedan tip u njoj definisan u tekućem crate-u.
- Inače bi dva crate-a mogla da implementiraju istu osobinu za isti tip, pa Rust ne bi znao koju da koristi.
- Dozvoljeno: `impl Summary for Vec<i32>`, `impl Display for Tweet`.
- Nije dozvoljeno: `impl Display for Vec<i32>`.

Osobine: podrazumevane implementacije

- Osobine mogu imati podrazumevane metode za ponašanje.
- Tipovi mogu koristiti ili prilagoditi podrazumevane metode.
  - Iz prilagođene metode ne možemo pozvati njenu podrazumevanu verziju.
- Podrazumevane metode mogu pozivati druge metode iste osobine.
- Smanjuje potrebu za pisanjem redundantnog koda.

Osobine: podrazumevane implementacije, primer

primeri/traits-lifetimes-errors/src/bin/default.rs

```
// Define a trait with default and required methods
pub trait Summary {
    fn summarize_author(&self) -> String; // Required method

    // Default implementation for the summarize method
    fn summarize(&self) -> String {
        format!("(Read more from {}...)", self.summarize_author())
    }
}

// Struct implementing only the required method
```

```rust
pub struct Tweet {
    pub username: String,
    pub content: String,
    pub reply: bool,
    pub retweet: bool,
}

impl Summary for Tweet {
    fn summarize_author(&self) -> String {
        format!("@{}", self.username)
    }
    // Uses the default summarize method from Summary trait
}

// Struct implementing both required and default methods
pub struct NewsArticle {
    pub headline: String,
    pub location: String,
    pub author: String,
    pub content: String,
}

impl Summary for NewsArticle {
    fn summarize_author(&self) -> String {
        format!("{}", self.author)
    }

    // Overrides the default summarize method
    fn summarize(&self) -> String {
        format!("{} - by {}, {}", self.headline, self.author,
         ↪  self.location)
    }
}

fn main() {
    let tweet = Tweet {
        username: String::from("horse_ebooks"),
        content: String::from("of course, as you probably already know,
 ↪  people"),
        reply: false,
        retweet: false,
    };

    let article = NewsArticle {
        headline: String::from("Penguins win the Stanley Cup
 ↪  Championship!"),
        location: String::from("Pittsburgh, PA, USA"),
        author: String::from("Iceburgh"),
        content: String::from(
            "The Pittsburgh Penguins once again are the best hockey team
             ↪  in the NHL.",
        ),
    };

    println!("1 new tweet: {}", tweet.summarize());
    println!("New article available! {}", article.summarize());
    println!("Author of the tweet: {}", tweet.summarize_author());
    println!("Author of the article: {}", article.summarize_author());
```

```
}
```

Izlaz

```
1 new tweet: (Read more from @horse_ebooks...)
New article available! Penguins win the Stanley Cup Championship! - by
 ↪  Iceburgh, Pittsburgh, PA, USA
Author of the tweet: @horse_ebooks
Author of the article: Iceburgh
```

Osobine: parametri

- `impl Trait` omogućava prihvatanje bilo kog tipa koji implementira osobinu.
- `T: Trait` (ograničenje osobine, *trait bound*) je duži oblik istog zapisa.
  - Sa `impl Trait` parametri mogu da budu različitih tipova, a sa `T` svi parametri tog tipa moraju da budu isti tip.
- Parametar može da zahteva više osobina odjednom, spojenih sa `+`.
- `where` klauzula povećava čitljivost.

Osobine: parametri, primer

primeri/traits-lifetimes-errors/src/bin/parameters.rs

```rust
use std::fmt::{Debug, Display};

// Define a Summary trait with a default implementation
pub trait Summary {
    fn summarize(&self) -> String {
        String::from("(Read more...)")
    }
}

// Implement Summary trait for NewsArticle
#[derive(Clone, Debug)]
pub struct NewsArticle {
    pub headline: String,
    pub location: String,
    pub author: String,
    pub content: String,
}

impl Summary for NewsArticle {}

// Implement Summary and Display traits for Tweet
pub struct Tweet {
    pub username: String,
    pub content: String,
}

impl Summary for Tweet {
    fn summarize(&self) -> String {
        format!("{}: {}", self.username, self.content)
    }
}

impl Display for Tweet {
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
        write!(f, "@{}: {}", self.username, self.content)
```

```
    }
}

// Function using impl Trait for single parameter
pub fn notify_simple(item: &impl Summary) {
    println!("Breaking news! {}", item.summarize());
}

// Function using trait bound syntax for a single parameter
pub fn notify<T: Summary>(item: &T) {
    println!("Breaking news! {}", item.summarize());
}

// Multiple parameters of possibly different types, using impl Trait
pub fn notify_multiple(item1: &impl Summary, item2: &impl Summary) {
    println!("Item 1: {}", item1.summarize());
    println!("Item 2: {}", item2.summarize());
}

// Multiple parameters of the same type, enforced by a trait bound
pub fn notify_same_type<T: Summary>(item1: &T, item2: &T) {
    println!("Item 1: {}", item1.summarize());
    println!("Item 2: {}", item2.summarize());
}

// Function requiring multiple trait bounds using + syntax
pub fn notify_with_display(item: &(impl Summary + Display)) {
    println!("Notified: {}", item);
}

// Function with multiple bounds using where clause for readability
pub fn multiple_bounds_where<T, U>(t: &T, u: &U)
where
    T: Summary + Display,
    U: Clone + Debug,
{
    println!("T: {}, U: {:?}", t.summarize(), u);
}

fn main() {
    let article = NewsArticle {
        headline: String::from("Penguins win the Stanley Cup
 ↪  Championship!"),
        location: String::from("Pittsburgh, PA, USA"),
        author: String::from("Iceburgh"),
        content: String::from(
            "The Pittsburgh Penguins once again are the best hockey team
             ↪  in the NHL.",
        ),
    };

    let tweet = Tweet {
        username: String::from("horse_ebooks"),
        content: String::from("of course, as you probably already know,
 ↪  people"),
    };

    // Demonstrate various function calls
```

```
    notify_simple(&article); // uses impl Trait
    notify(&tweet); // uses trait bound syntax
    notify_multiple(&article, &tweet); // different types using impl Trait
    notify_same_type(&tweet, &tweet); // same type enforced with trait
 ↪  bound
    notify_with_display(&tweet); // multiple trait bounds using + syntax

    // Example using where clause for multiple bounds
    multiple_bounds_where(&tweet, &article);
}
```

Izlaz

```
Breaking news! (Read more...)
Breaking news! horse_ebooks: of course, as you probably already know,
 ↪  people
Item 1: (Read more...)
Item 2: horse_ebooks: of course, as you probably already know, people
Item 1: horse_ebooks: of course, as you probably already know, people
Item 2: horse_ebooks: of course, as you probably already know, people
Notified: @horse_ebooks: of course, as you probably already know, people
T: horse_ebooks: of course, as you probably already know, people, U:
 ↪  NewsArticle { headline: "Penguins win the Stanley Cup Championship!",
 ↪  location: "Pittsburgh, PA, USA", author: "Iceburgh", content: "The
 ↪  Pittsburgh Penguins once again are the best hockey team in the NHL." }
```

Osobine: povratne vrednosti

- `impl Trait` skriva konkretan tip povratne vrednosti.
  - Na primer, `-> impl Trait` ostavlja slobodu da se konkretan tip kasnije promeni.
- **Ograničeno na jedan konkretan povratni tip.**
- Za različite tipove koji implementiraju istu osobinu koristi se `Box<dyn Trait>`.

Osobine: `dyn Trait`

- `dyn Trait` je zamena za bilo koji tip koji implementira osobinu. Pravi se pomoću pokazivača, na primer `Box<dyn Trait>` ili `&dyn Trait`.
- Tako različiti tipovi koji implementiraju istu osobinu mogu da stanu u isti vektor.
- Kod `impl Trait` i `T: Trait` kompajler zna konkretan tip, pa metodu bira pri kompajliranju (statički izbor).
- Kod `dyn Trait` kompajler ne zna konkretan tip, pa se metoda bira tek pri izvršavanju (dinamički izbor).
  - To ima cenu: traženje metode pri izvršavanju i manje optimizacija.

Osobine: povratne vrednosti, primer

primeri/traits-lifetimes-errors/src/bin/return_val.rs

```
// Define the Summary trait with a summarize method
pub trait Summary {
    fn summarize(&self) -> String;
}

// Implement Summary for Tweet
pub struct Tweet {
    pub username: String,
    pub content: String,
    pub reply: bool,
```

```
    pub retweet: bool,
}

impl Summary for Tweet {
    fn summarize(&self) -> String {
        format!("@{}: {}", self.username, self.content)
    }
}

// Implement Summary for NewsArticle
pub struct NewsArticle {
    pub headline: String,
    pub location: String,
    pub author: String,
    pub content: String,
}

impl Summary for NewsArticle {
    fn summarize(&self) -> String {
        format!("{} - by {}, {}", self.headline, self.author,
        ↪  self.location)
    }
}

// Function returning a single type implementing Summary (Tweet)
fn returns_summarizable() -> impl Summary {
    Tweet {
        username: String::from("horse_ebooks"),
        content: String::from("of course, as you probably already know,
 ↪  people"),
        reply: false,
        retweet: false,
    }
}

// impl Summary can't return different types (nocompile_return_either.rs),
// but returning different types works with Box<dyn Summary>
fn returns_summarizable_trait_object(switch: bool) -> Box<dyn Summary> {
    if switch {
        Box::new(NewsArticle {
            headline: String::from("Penguins win the Stanley Cup
 ↪  Championship!"),
            location: String::from("Pittsburgh, PA, USA"),
            author: String::from("Iceburgh"),
            content: String::from(
                "The Pittsburgh Penguins once again are the best hockey
                 ↪  team in the NHL.",
            ),
        })
    } else {
        Box::new(Tweet {
            username: String::from("horse_ebooks"),
            content: String::from("of course, as you probably already
 ↪  know, people"),
            reply: false,
            retweet: false,
        })
    }
```

```
}

fn main() {
    // Using the impl Trait function
    let tweet_summary = returns_summarizable();
    println!("Tweet summary: {}", tweet_summary.summarize());

    // Using the trait object function to handle different types
    let article_summary = returns_summarizable_trait_object(true);
    let tweet_summary2 = returns_summarizable_trait_object(false);
    println!("Article summary: {}", article_summary.summarize());
    println!("Another tweet summary: {}", tweet_summary2.summarize());
}
```

Izlaz

```
Tweet summary: @horse_ebooks: of course, as you probably already know,
 ↪  people
Article summary: Penguins win the Stanley Cup Championship! - by Iceburgh,
 ↪  Pittsburgh, PA, USA
Another tweet summary: @horse_ebooks: of course, as you probably already
 ↪  know, people
```

Osobine: `impl Trait` vraća jedan tip (greška)

primeri/traits-lifetimes-errors/src/bin/nocompile_return_either.rs

```
pub trait Summary {
    fn summarize(&self) -> String;
}

pub struct Tweet {
    pub content: String,
}

impl Summary for Tweet {
    fn summarize(&self) -> String {
        format!("Tweet: {}", self.content)
    }
}

pub struct NewsArticle {
    pub headline: String,
}

impl Summary for NewsArticle {
    fn summarize(&self) -> String {
        format!("Article: {}", self.headline)
    }
}

// Returning multiple types - this does NOT compile
fn returns_either(switch: bool) -> impl Summary {
    if switch {
        NewsArticle {
            headline: String::from("Penguins win the Stanley Cup!"),
        }
    } else {
```

```
        Tweet {
            content: String::from("of course, as you probably know"),
        }
    }
}

fn main() {
    println!("{}", returns_either(true).summarize());
}
```

Izlaz

```
error[E0308]: `if` and `else` have incompatible types
  --> traits-lifetimes-errors/src/bin/nocompile_return_either.rs:32:9
   |
27 | /       if switch {
28 | | /         NewsArticle {
29 | | |             headline: String::from("Penguins win the Stanley
 ↪  Cup!"),
30 | | |         }
   | | |_________- expected because of this
31 | |       } else {
32 | | /         Tweet {
33 | | |             content: String::from("of course, as you probably
 ↪  know"),
34 | | |         }
   | | |_________^ expected `NewsArticle`, found `Tweet`
35 | |       }
   | |_______- `if` and `else` have incompatible types
   |
help: you could change the return type to be a boxed trait object
   |
26 - fn returns_either(switch: bool) -> impl Summary {
26 + fn returns_either(switch: bool) -> Box<dyn Summary> {
   |
help: if you change the return type to expect trait objects, box the
 ↪  returned expressions
   |
28 ~         Box::new(NewsArticle {
29 |             headline: String::from("Penguins win the Stanley Cup!"),
30 ~         })
31 |     } else {
32 ~         Box::new(Tweet {
33 |             content: String::from("of course, as you probably know"),
34 ~         })
   |

For more information about this error, try `rustc --explain E0308`.
error: could not compile `traits-lifetimes-errors` (bin
 ↪  "nocompile_return_either") due to 1 previous error
```

Osobine: povratne vrednosti, objašnjenje

- `returns_summarizable` uvek vraća `Tweet`, pa `impl Summary` radi.
- `returns_summarizable_trait_object` vraća `NewsArticle` ili `Tweet`, pa koristi `Box<dyn Summary>`.
- Isti kod sa `impl Summary` kompajler odbija: `if` i `else` imaju različite tipove.

Osobine: uslovna implementacija metoda

- Moguće je implementirati određene metode ukoliko tip poseduje određenu osobinu.
- Provere se vrše pri kompajliranju koda.

Osobine: primer uslovne implementacije

primeri/traits-lifetimes-errors/src/bin/cond_impl.rs

```rust
use std::fmt::Display;

// Define a generic struct Pair<T>
struct Pair<T> {
    x: T,
    y: T,
}

// Always implement the new function for Pair<T>
impl<T> Pair<T> {
    fn new(x: T, y: T) -> Self {
        Self { x, y }
    }
}

// Conditionally implement cmp_display if T: Display + PartialOrd
impl<T: Display + PartialOrd> Pair<T> {
    fn cmp_display(&self) {
        if self.x >= self.y {
            println!("The largest member is x = {}", self.x);
        } else {
            println!("The largest member is y = {}", self.y);
        }
    }
}

fn main() {
    let pair = Pair::new(3, 5);
    pair.cmp_display(); // Works because i32 implements Display and
 ↪  PartialOrd

    let float_pair = Pair::new(4.2, 1.8);
    float_pair.cmp_display(); // Works because f64 implements Display and
 ↪  PartialOrd

    // to_string comes from the blanket impl
    // `impl<T: Display> ToString for T`
    let num_str = 3.to_string();
    println!("Converted integer to string: {}", num_str);
}
```

Izlaz

```
The largest member is y = 5
The largest member is x = 4.2
Converted integer to string: 3
```

Osobine: blanket implementacija

- Blanket implementacija je implementacija osobine za svaki tip koji ispunjava zadati uslov.

- Često se koristi u standardnoj biblioteci.
  - Na primer, `ToString` je implementirana za svaki tip koji implementira `Display`.
  - Zato `to_string` možemo da pozovemo nad svakim takvim tipom.

Osobine: blanket implementacija, primer

primeri/traits-lifetimes-errors/src/bin/blanket.rs

```rust
use std::fmt::Display;

// Define a custom trait called Printable
trait Printable {
    fn print(&self);
}

// Blanket implementation: Printable for every type that implements
 ↪  Display
impl<T: Display> Printable for T {
    fn print(&self) {
        println!("{}", self);
    }
}

// Test struct without implementing Printable directly
struct User {
    name: String,
}

// Implement Display for User so it can use Printable automatically
impl Display for User {
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
        write!(f, "User: {}", self.name)
    }
}

fn main() {
    // Using Printable for types with Display
    let number = 42;
    let user = User {
        name: String::from("Alice"),
    };

    // `print` from Printable works thanks to the blanket implementation
    number.print(); // Works because i32 implements Display
    user.print(); // Works because User implements Display
}
```

Izlaz

```
42
User: Alice
```

Osobine: blanket implementacija, objašnjenje

- `impl<T: Display> Printable for T` implementira `Printable` za svaki tip koji implementira `Display`.
- `number.print()` radi jer `i32` implementira `Display`.
- `User` implementira samo `Display`, a `print` dobija automatski.

Životni vekovi

Životni vekovi (*Lifetimes*)

- Životni vekovi u Rust-u osiguravaju da reference ostanu važeće tokom korišćenja.
- Cilj životnih vekova je da spreče nevažeće reference.
- U većini slučajeva, vekovi se automatski zaključuju.
- Životni vekovi moraju biti eksplicitni kada reference imaju složene odnose.
- Rust-ov borrow-checker detektuje probleme sa referencama tokom kompajliranja.

Životni vekovi: referenca nadživi vrednost (greška)

primeri/traits-lifetimes-errors/src/bin/nocompile_lifetime_a.rs

```
#[rustfmt::skip]
fn main() {
    let r;                // ---------+-- 'a
                          //          |
    {                     //          |
        let x = 5;        // -+-- 'b  |
        r = &x;           //  |       |
    }                     // -+       |
                          //          |
    println!("r: {r}");   //          |
                          // ---------+
}
```

Izlaz

```
error[E0597]: `x` does not live long enough
  --> traits-lifetimes-errors/src/bin/nocompile_lifetime_a.rs:7:13
   |
 6 |         let x = 5;        // -+-- 'b  |
   |             - binding `x` declared here
 7 |         r = &x;           //  |       |
   |             ^^ borrowed value does not live long enough
 8 |     }                     // -+       |
   |     - `x` dropped here while still borrowed
 9 |                           //          |
10 |     println!("r: {r}");   //          |
   |                    - borrow later used here

For more information about this error, try `rustc --explain E0597`.
error: could not compile `traits-lifetimes-errors` (bin
 ↪  "nocompile_lifetime_a") due to 1 previous error
```

Životni vekovi: ispravan doseg referenci

primeri/traits-lifetimes-errors/src/bin/lifetime_b.rs

```
#[rustfmt::skip]
fn main() {
    let x = 5;            // ----------+-- 'b
                          //           |
    let r = &x;           // --+-- 'a  |
                          //   |       |
    println!("r: {r}");   //   |       |
                          // --+       |
```

```
                            // ----------+
}
```

Izlaz

```
r: 5
```

Životni vekovi: anotacije

- Anotacije životnih vekova opisuju odnose vekova referenci **bez menjanja trajanja.**
- Anotacije omogućavaju funkcijama da prihvate reference sa bilo kojim vekom.
- Imena vekova počinju apostrofom `'`.
- Najčešće korišćeno ime za vek je `'a`.
- Sintaksa:

```
&i32        // a reference
&'a i32     // a reference with an explicit lifetime
&'a mut i32 // a mutable reference with an explicit lifetime
```

Životni vekovi: u potpisima funkcija

- Navode se u potpisima funkcija, kao jedan od generičkih parametara.
- Ako parametri i rezultat dele isti `'a`, rezultat važi koliko i najkraće živeći parametar.

Životni vekovi: funkcija bez anotacije (greška)

primeri/traits-lifetimes-errors/src/bin/nocompile_function_a.rs

```
fn longest(x: &str, y: &str) -> &str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}

fn main() {
    let string1 = String::from("This is a long string");

    {
        let string2 = String::from("Short");
        let result = longest(string1.as_str(), string2.as_str());
        println!("The longest string is '{}'", result);
    }
}
```

Izlaz

```
error[E0106]: missing lifetime specifier
 --> traits-lifetimes-errors/src/bin/nocompile_function_a.rs:1:33
  |
1 | fn longest(x: &str, y: &str) -> &str {
  |               ----     ----     ^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the
 ↪  signature does not say whether it is borrowed from `x` or `y`
help: consider introducing a named lifetime parameter
  |
```

```
1 | fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
  |           ++++     ++          ++          ++

For more information about this error, try `rustc --explain E0106`.
error: could not compile `traits-lifetimes-errors` (bin
 ↪  "nocompile_function_a") due to 1 previous error
```

Životni vekovi: funkcija sa anotacijom

primeri/traits-lifetimes-errors/src/bin/function_b.rs

```rust
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}

fn main() {
    let string1 = String::from("This is a long string");

    {
        let string2 = String::from("Short");
        let result = longest(string1.as_str(), string2.as_str());
        println!("The longest string is '{}'", result);
    }
}
```

Izlaz

```
The longest string is 'This is a long string'
```

Životni vekovi: rezultat nadživi argument (greška)

primeri/traits-lifetimes-errors/src/bin/nocompile_function_c.rs

```rust
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}

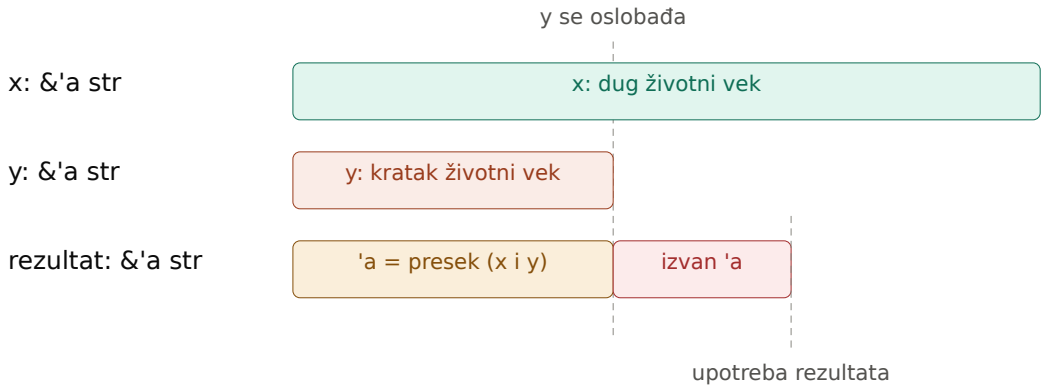
fn main() {
    let string1 = String::from("This is a long string");
    let result;
    {
        let string2 = String::from("xyz");
        result = longest(string1.as_str(), string2.as_str());
    } // `string2` is dropped here
    // Error: `result` is used after `string2` is dropped
    println!("The longest string is '{}'", result);
}
```

Izlaz

```
error[E0597]: `string2` does not live long enough
  --> traits-lifetimes-errors/src/bin/nocompile_function_c.rs:14:44
   |
13 |         let string2 = String::from("xyz");
   |             ------- binding `string2` declared here
14 |         result = longest(string1.as_str(), string2.as_str());
   |                                             ^^^^^^^ borrowed value
 ↪  does not live long enough
15 |     } // `string2` is dropped here
   |     - `string2` dropped here while still borrowed
16 |     // Error: `result` is used after `string2` is dropped
17 |     println!("The longest string is '{}'", result);
   |                                             ------ borrow later used
 ↪  here

For more information about this error, try `rustc --explain E0597`.
error: could not compile `traits-lifetimes-errors` (bin
 ↪  "nocompile_function_c") due to 1 previous error
```

Životni vekovi: jedan životni vek 'a



Slika 1: Greška: rezultat je ograničen kraćim argumentom, a koristi se izvan 'a

Životni vekovi: dva životna veka, primer

primeri/traits-lifetimes-errors/src/bin/lifetime_two_params.rs

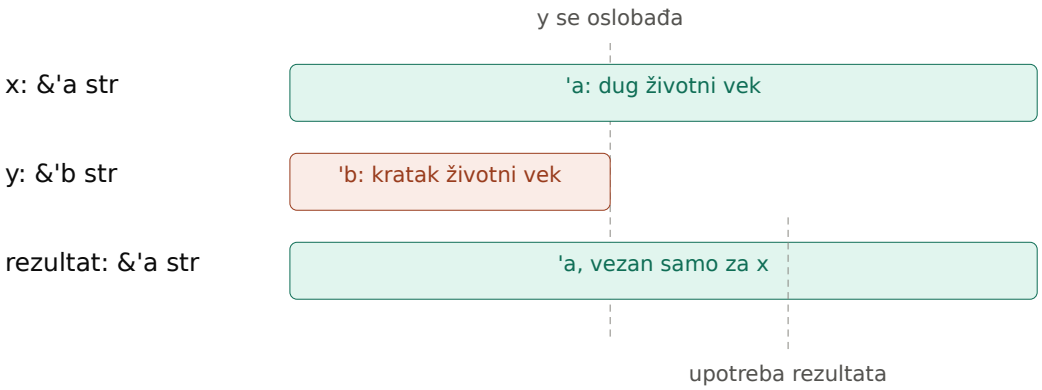
```rust
// The result depends only on `x`, so it gets the lifetime of `x`
fn first<'a, 'b>(x: &'a str, y: &'b str) -> &'a str {
    println!("Ignored: {}", y);
    x
}

fn main() {
    let string1 = String::from("This is a long string");
    let result;
    {
        let string2 = String::from("xyz");
        result = first(string1.as_str(), string2.as_str());
    } // `string2` is dropped here
    // OK: `result` is tied only to `string1`
    println!("The first string is '{}'", result);
}
```

Izlaz

```
Ignored: xyz
The first string is 'This is a long string'
```

Životni vekovi: dva životna veka



Slika 2: Prolazi: rezultat je vezan samo za prvi argument, pa je upotreba unutar `'a`

Životni vekovi u strukturama

Životni vekovi: struktura koja drži referencu

- Ako je polje strukture referenca, struktura mora da navede životni vek.
  - Bez toga kompajler prijavljuje grešku `E0106` (nedostaje specifikator životnog veka).
- Struktura ne sme da nadživi vrednost na koju njeno polje pokazuje.
- Ime životnog veka je proizvoljno; `'a` je samo uobičajeno. Ovde koristimo opisna imena.

Životni vekovi: struktura koja drži referencu, primer

primeri/traits-lifetimes-errors/src/bin/lifetime_struct.rs

```rust
// A struct that holds a reference needs a lifetime parameter.
// 'text: the struct cannot outlive the text its field points to.
struct Excerpt<'text> {
    part: &'text str,
}

fn main() {
    // `novel` owns the text
    let novel = String::from("Call me Ishmael. Some years ago...");
    // `first_sentence` borrows from `novel`
    let first_sentence = novel.split('.').next().unwrap();
    // `excerpt` holds that reference, so it is valid while `novel` exists
    let excerpt = Excerpt {
        part: first_sentence,
    };
    println!("{}", excerpt.part);
}
```

Izlaz

```
Call me Ishmael
```

Životni vekovi: struktura nadživi tekst (greška)

primeri/traits-lifetimes-errors/src/bin/nocompile_lifetime_struct.rs

```rust
struct Excerpt<'text> {
    part: &'text str,
}

fn main() {
    let excerpt;
    {
        let novel = String::from("Call me Ishmael. Some years ago...");
        excerpt = Excerpt { part: &novel };
    } // `novel` is dropped here, but `excerpt` still points to it
    // Error: `novel` does not live long enough
    println!("{}", excerpt.part);
}
```

Izlaz

```
error[E0597]: `novel` does not live long enough
  --> traits-lifetimes-errors/src/bin/nocompile_lifetime_struct.rs:9:35
   |
 8 |         let novel = String::from("Call me Ishmael. Some years
 ↪  ago...");
   |             ----- binding `novel` declared here
 9 |         excerpt = Excerpt { part: &novel };
   |                                   ^^^^^^ borrowed value does not live
 ↪  long enough
10 |     } // `novel` is dropped here, but `excerpt` still points to it
   |     - `novel` dropped here while still borrowed
11 |     // Error: `novel` does not live long enough
12 |     println!("{}", excerpt.part);
   |                     ------------ borrow later used here

For more information about this error, try `rustc --explain E0597`.
error: could not compile `traits-lifetimes-errors` (bin
 ↪  "nocompile_lifetime_struct") due to 1 previous error
```

Životni vekovi: struktura sa referencom, objašnjenje

- `'text` u `struct Excerpt<'text>` je životni vek teksta na koji pokazuje polje `part`.
- `novel` je vlasnik teksta, a `excerpt` samo pozajmljuje njegov deo.
- U grešci `excerpt` nadživi `novel`, pa kompajler prijavljuje `E0597`.

Životni vekovi: dva veka u strukturi

primeri/traits-lifetimes-errors/src/bin/lifetime_two_fields.rs

```rust
// Source: https://stackoverflow.com/questions/29861388/
 ↪  when-is-it-useful-to-define-multiple-lifetimes-in-a-struct
static ZERO: i32 = 0;

struct Foo<'a, 'b> {
    x: &'a i32,
    y: &'b i32,
}

fn get_x_or_zero_ref<'a, 'b>(x: &'a i32, y: &'b i32) -> &'a i32 {
```

```
    if *x > *y {
        return x;
    } else {
        return &ZERO;
    }
}

fn main() {
    let x = 1;
    let v;
    {
        let y = 2;
        let f = Foo { x: &x, y: &y };
        v = get_x_or_zero_ref(&f.x, &f.y);
    }
    println!("{}", *v);
}
```

Izlaz

```
0
```

Životni vekovi: jedan vek za dva polja (greška)

primeri/traits-lifetimes-errors/src/bin/nocompile_lifetime_two_fields.rs

```
// Source: https://stackoverflow.com/questions/29861388/
 ↪  when-is-it-useful-to-define-multiple-lifetimes-in-a-struct
static ZERO: i32 = 0;

struct Foo<'a> {
    x: &'a i32,
    y: &'a i32,
}

fn get_x_or_zero_ref<'a, 'b>(x: &'a i32, y: &'b i32) -> &'a i32 {
    if *x > *y {
        return x;
    } else {
        return &ZERO;
    }
}

fn main() {
    let x = 1;
    let v;
    {
        let y = 2;
        let f = Foo { x: &x, y: &y };
        v = get_x_or_zero_ref(&f.x, &f.y);
    }
    // Error: `y` does not live long enough
    println!("{}", *v);
}
```

Izlaz

```
error[E0597]: `y` does not live long enough
  -->
 ↪  traits-lifetimes-errors/src/bin/nocompile_lifetime_two_fields.rs:22:33
   |
21 |         let y = 2;
   |             - binding `y` declared here
22 |         let f = Foo { x: &x, y: &y };
   |                                 ^^ borrowed value does not live long
 ↪  enough
23 |         v = get_x_or_zero_ref(&f.x, &f.y);
24 |     }
   |     - `y` dropped here while still borrowed
25 |     // Error: `y` does not live long enough
26 |     println!("{}", *v);
   |                    -- borrow later used here

For more information about this error, try `rustc --explain E0597`.
error: could not compile `traits-lifetimes-errors` (bin
 ↪  "nocompile_lifetime_two_fields") due to 1 previous error
```

Životni vekovi: dva veka u strukturi, objašnjenje

- Dva životna veka su korisna kad polja pokazuju na vrednosti koje važe različito dugo, a rezultat ne pokazuje na kraće živeću.
- Sa `Foo<'a>` oba polja dele isti `'a`, pa `v` ne sme da se koristi posle oslobađanja `y`: kompajler prijavljuje `E0597`.
- Sa `Foo<'a, 'b>` polja su nezavisna, pa `v` sme da se koristi i posle oslobađanja `y`. Zato i `get_x_or_zero_ref` ima dva životna veka.

Životni vekovi u metodama

Automatski životni vekovi

- To je skup pravila kompajlera za automatsko dodeljivanje životnih vekova.
  - Smanjuje potrebu za eksplicitnim anotacijama.
- Pravila:
  - **Pravilo 1**: Svaki izostavljen životni vek u parametrima postaje zaseban životni vek.
  - **Pravilo 2**: Ako postoji samo jedan ulazni životni vek, on se dodeljuje svim izlaznim referencama.
  - **Pravilo 3**: Za metode, ako ima više životnih vekova, životni vek `self` se dodeljuje svim izlaznim referencama.
- Ako pravila ne mogu da odrede sve životne vekove, potrebne su ručne anotacije.

Automatski životni vekovi: primer

primeri/traits-lifetimes-errors/src/bin/elision_functions.rs

```rust
// One reference parameter: rules 1 and 2 give the output its lifetime
// Inferred: fn first_word<'a>(s: &'a str) -> &'a str
fn first_word(s: &str) -> &str {
    s.split(' ').next().unwrap()
}

// Two reference parameters: rule 1 gives them separate lifetimes and no
 ↪  rule
```

```
// sets the output, so the annotation is required
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}

fn main() {
    println!("{}", first_word("Call me Ishmael."));
    println!("{}", longest("long string", "short"));
}
```

Izlaz

```
Call
long string
```

Životni vekovi: metode

- Implementacija metoda sa životnim vekom koristi sličnu sintaksu kao generički tipovi.
- Životni vek za polja strukture navodi se posle `impl`, jer je deo njenog tipa.
- U `impl` bloku, životni vekovi metoda mogu biti vezani za polja strukture ili nezavisni.
- Rust često automatski dodaje životni vek, pa eksplicitna anotacija nije uvek potrebna.
- Metoda pozajmljuje samu strukturu (`&self` ili `&mut self`). Ta pozajmica ima svoj životni vek, različit od životnog veka polja.
  - Počinje pozivom metode. Ako je rezultat vezan za pozajmicu, traje do poslednjeg korišćenja rezultata; inače traje samo koliko i poziv.

Životni vekovi: metode, primer

primeri/traits-lifetimes-errors/src/bin/elision_methods.rs

```
struct Excerpt<'text> {
    part: &'text str,
}

impl<'text> Excerpt<'text> {
    // Rules 1 and 3: the result gets the lifetime of `&self`
    // Inferred: fn announce<'borrow, 'arg>(
    //     &'borrow self, announcement: &'arg str) -> &'borrow str
    fn announce(&self, announcement: &str) -> &str {
        println!("Announcement: {}", announcement);
        self.part
    }
}

fn main() {
    let novel = String::from("Call me Ishmael.");
    let excerpt = Excerpt { part: &novel };
    println!("{}", excerpt.announce("Here's an excerpt"));
}
```

Izlaz

```
Announcement: Here's an excerpt
Call me Ishmael.
```

Životni vekovi: metode, objašnjenje

- Komentari u kodu prikazuju životne vekove koje kompajler sam zaključuje.
- `'text` je životni vek teksta na koji pokazuje `part`.
- `'borrow` je životni vek pozajmice `&self`.

Životni vekovi: `'text` i `'borrow`, primer

primeri/traits-lifetimes-errors/src/bin/lifetime_text_and_borrow.rs

```
struct Excerpt<'text> {
    part: &'text str,
}

impl<'text> Excerpt<'text> {
    // The result is tied to the text
    fn text_part(&self) -> &'text str {
        self.part
    }

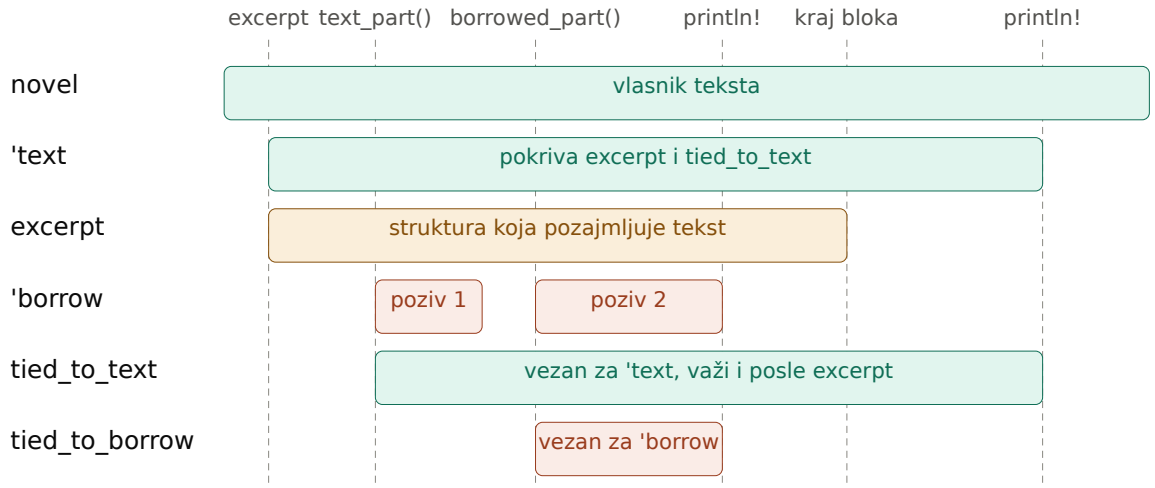
    // The result is tied to the borrow of `self`
    // `&self` holds two lifetimes ('borrow and 'text), so rule 2 does not
     ↪  apply.
    // Rules 1 and 3: the result gets the lifetime of `&self`
    // Inferred: fn borrowed_part<'borrow>(&'borrow self) -> &'borrow str
    fn borrowed_part(&self) -> &str {
        self.part
    }
}

fn main() {
    let novel = String::from("Call me Ishmael.");
    let tied_to_text;
    {
        let excerpt = Excerpt { part: &novel };
        tied_to_text = excerpt.text_part();
        let tied_to_borrow = excerpt.borrowed_part();
        println!("{}", tied_to_borrow);
    } // `excerpt` is dropped here, `novel` still exists
    println!("{}", tied_to_text);
}
```

Izlaz

```
Call me Ishmael.
Call me Ishmael.
```

Životni vekovi: `'text` i `'borrow`



Slika 3: Prolazi: svaka pozajmica je unutar života `excerpt`, a `excerpt` je unutar `'text`

Životni vekovi: `'text` i `'borrow` (greška)

primeri/traits-lifetimes-errors/src/bin/nocompile_lifetime_text_and_borrow.rs

```
struct Excerpt<'text> {
    part: &'text str,
}

impl<'text> Excerpt<'text> {
    // Inferred: fn borrowed_part<'borrow>(&'borrow self) -> &'borrow str
    fn borrowed_part(&self) -> &str {
        self.part
    }
}

fn main() {
    let novel = String::from("Call me Ishmael.");
    let tied_to_borrow;
    {
        let excerpt = Excerpt { part: &novel };
        tied_to_borrow = excerpt.borrowed_part();
        // `excerpt` is dropped at the end of this block, `novel` still
        ↪  exists
    }
    // Error: the borrow of `excerpt` cannot reach this line
    println!("{}", tied_to_borrow);
}
```

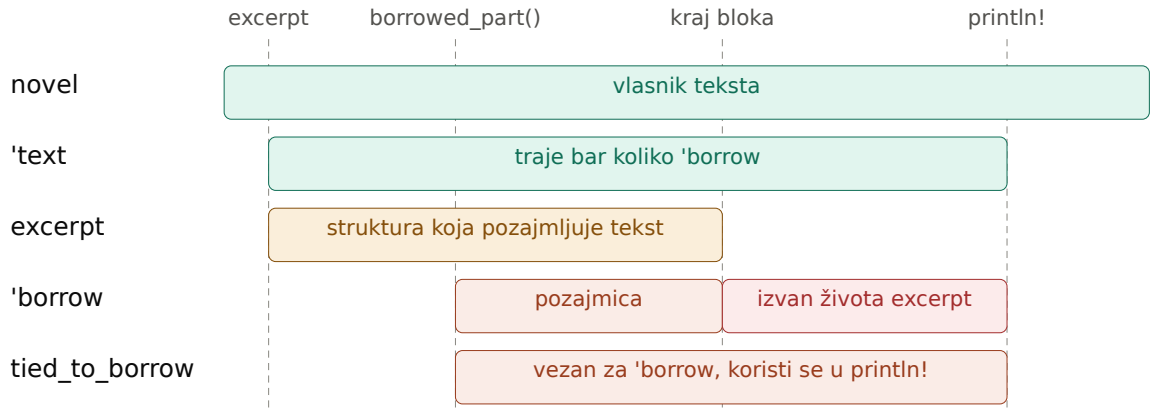
Izlaz

```
error[E0597]: `excerpt` does not live long enough
  --> traits-lifetimes-errors/src/bin/
 ↪  nocompile_lifetime_text_and_borrow.rs:17:26
   |
16 |         let excerpt = Excerpt { part: &novel };
   |             ------- binding `excerpt` declared here
17 |         tied_to_borrow = excerpt.borrowed_part();
```

```
   |                          ^^^^^^^ borrowed value does not live long
 ↪  enough
18 |         // `excerpt` is dropped at the end of this block, `novel`
 ↪  still exists
19 |     }
   |     - `excerpt` dropped here while still borrowed
20 |     // Error: the borrow of `excerpt` cannot reach this line
21 |     println!("{}", tied_to_borrow);
   |                    -------------- borrow later used here

For more information about this error, try `rustc --explain E0597`.
error: could not compile `traits-lifetimes-errors` (bin
 ↪  "nocompile_lifetime_text_and_borrow") due to 1 previous error
```

Životni vekovi: pozajmica nadživi strukturu



Slika 4: Greška: upotreba rezultata traži da `'borrow` traje duže od `excerpt`

Životni vekovi: `'text` i `'borrow`, objašnjenje

- `'text` je životni vek teksta van strukture, a `'borrow` životni vek jedne pozajmice strukture: svaki poziv ima svoju.
- `'text` mora da traje bar koliko `'borrow`, jer pozajmica strukture ne sme da nadživi tekst na koji struktura pokazuje. Kompajler taj uslov sam zaključuje iz tipa `&'borrow Excerpt<'text>`.
- Zato se `self.part` može vratiti i kao `&'text str` i kao `&'borrow str`.
- Polje koje struktura sama poseduje (na primer `String`) može da se vrati samo kao `&'borrow str`.

Životni vekovi: rezultat i pozajmica

- Struktura koja pozajmljuje tekst često ima metode koje vraćaju njegove delove, na primer parser ili tokenizer.
- Ako takva metoda menja strukturu (`&mut self`), važno je za šta je rezultat vezan: od toga zavisi da li možemo da držimo rezultat i ponovo pozovemo metodu.

Životni vekovi: rezultat vezan za tekst, primer

`primeri/traits-lifetimes-errors/src/bin/lifetime_lexer.rs`

```
// A struct that borrows a text and hands out pieces of it.
struct Lexer<'text> {
    input: &'text str,
}

impl<'text> Lexer<'text> {
    // The result is tied to the text ('text), not to the borrow of
     ↪  `self`,
    // so earlier results stay valid after the next call.
    fn next_word(&mut self) -> &'text str {
        let input = self.input.trim_start();
        let end = input.find(' ').unwrap_or(input.len());
        let (word, rest) = input.split_at(end);
        self.input = rest;
        word
    }
}

fn main() {
    let text = String::from("let x = 5");
    let mut lexer = Lexer { input: &text };

    let first = lexer.next_word();
    let second = lexer.next_word();
    println!("{} {}", first, second);
}
```

Izlaz

```
let x
```

Životni vekovi: rezultat vezan za pozajmicu (greška)

primeri/traits-lifetimes-errors/src/bin/nocompile_lifetime_lexer.rs

```
struct Lexer<'text> {
    input: &'text str,
}

impl<'text> Lexer<'text> {
    // Elided: fn next_word<'borrow>(&'borrow mut self) -> &'borrow str
    // The result is tied to the borrow of `self`
    fn next_word(&mut self) -> &str {
        let input = self.input.trim_start();
        let end = input.find(' ').unwrap_or(input.len());
        let (word, rest) = input.split_at(end);
        self.input = rest;
        word
    }
}

fn main() {
    let text = String::from("let x = 5");
    let mut lexer = Lexer { input: &text };

    let first = lexer.next_word();
    // Error: `first` still borrows `lexer`
    let second = lexer.next_word();
```

```
    println!("{} {}", first, second);
}
```

Izlaz

```
error[E0499]: cannot borrow `lexer` as mutable more than once at a time
  --> traits-lifetimes-errors/src/bin/nocompile_lifetime_lexer.rs:23:18
   |
21 |     let first = lexer.next_word();
   |                 ----- first mutable borrow occurs here
22 |     // Error: `first` still borrows `lexer`
23 |     let second = lexer.next_word();
   |                  ^^^^^ second mutable borrow occurs here
24 |     println!("{} {}", first, second);
   |                        ----- first borrow later used here

For more information about this error, try `rustc --explain E0499`.
error: could not compile `traits-lifetimes-errors` (bin
 ↪  "nocompile_lifetime_lexer") due to 1 previous error
```

Životni vekovi: rezultat i pozajmica, objašnjenje

- Rezultat sa `-> &str` (izostavljeno) vezan je za pozajmicu `'borrow`: dok ga držimo, ne možemo ponovo da pozovemo metodu.
  - Dok se `first` koristi, mora da važi pozajmica `lexer`-a.
- Rezultat sa `-> &'text str` vezan je za tekst: ranije rezultate možemo da držimo i posle sledećeg poziva.
  - Dok se `first` koristi, ne mora da važi pozajmica `lexer`-a, nego samo pozajmica stringa `text`.
- Izostavljeni oblik: `fn next_word<'borrow>(&'borrow mut self) -> &'borrow str`.

Životni vekovi: `&mut self`

- `&mut self` pozajmljuje strukturu promenljivo.
- Ako rezultat metode nije vezan za tu pozajmicu, ona traje samo dok traje poziv, pa je struktura posle poziva opet slobodna.

Životni vekovi: `&mut self`, primer

primeri/traits-lifetimes-errors/src/bin/lifetime_mut_self.rs

```
struct Excerpt<'text> {
    part: &'text str,
}

impl<'text> Excerpt<'text> {
    // Elided: fn set_part<'borrow>(&'borrow mut self, new_part: &'text
     ↪  str)
    // The borrow of `self` ends when the call ends
    fn set_part(&mut self, new_part: &'text str) {
        self.part = new_part;
    }
}

fn main() {
    let mut excerpt = Excerpt {
        part: "Call me Ishmael.",
```

```
    };
    println!("before: {}", excerpt.part);

    excerpt.set_part("A new excerpt");
    println!("after the first call: {}", excerpt.part);

    excerpt.set_part("Another excerpt");
    println!("after the second call: {}", excerpt.part);
}
```

Izlaz

```
before: Call me Ishmael.
after the first call: A new excerpt
after the second call: Another excerpt
```

Životni vekovi: prekratka referenca (greška)

primeri/traits-lifetimes-errors/src/bin/nocompile_set_part_short_lived.rs

```
struct Excerpt<'text> {
    part: &'text str,
}

impl<'text> Excerpt<'text> {
    fn set_part(&mut self, new_part: &'text str) {
        self.part = new_part;
    }
}

fn main() {
    let novel = String::from("Call me Ishmael.");
    let mut excerpt = Excerpt { part: &novel };
    {
        let short_lived = String::from("A short-lived string");
        // Error: `short_lived` is dropped at the end of this block
        excerpt.set_part(&short_lived);
    }
    println!("{}", excerpt.part);
}
```

Izlaz

```
error[E0597]: `short_lived` does not live long enough
  --> traits-lifetimes-errors/src/bin/
 ↪  nocompile_set_part_short_lived.rs:17:26
   |
15 |         let short_lived = String::from("A short-lived string");
   |             ----------- binding `short_lived` declared here
16 |         // Error: `short_lived` is dropped at the end of this block
17 |         excerpt.set_part(&short_lived);
   |                          ^^^^^^^^^^^^ borrowed value does not live
 ↪  long enough
18 |     }
   |     - `short_lived` dropped here while still borrowed
19 |     println!("{}", excerpt.part);
   |                    ------------ borrow later used here
```

```
For more information about this error, try `rustc --explain E0597`.
error: could not compile `traits-lifetimes-errors` (bin
 ↪  "nocompile_set_part_short_lived") due to 1 previous error
```

Životni vekovi: `&mut self`, objašnjenje

- Izostavljeni životni vek pozajmice je nov i kratak (`'borrow`), nezavisan od `'text`.
- Novi tekst mora da traje bar koliko `'text`, inače kompajler prijavljuje `E0597`.

Životni vekovi: `&'text mut self`

- Ako životni vek pozajmice napišemo sami, `&'text mut self`, pišemo ograničenje: pozajmica `self` mora da važi od poziva do kraja `'text`.
- `'text` pokriva svako korišćenje strukture.
- Anotacija ne produžava život strukture, ali kompajler pozajmicu računa kao aktivnu dok god traje `'text`.

Životni vekovi: `&'text mut self` (greška)

primeri/traits-lifetimes-errors/src/bin/nocompile_set_part_from.rs

```rust
struct Excerpt<'text> {
    part: &'text str,
}

impl<'text> Excerpt<'text> {
    // `&'text mut self` borrows `self` from the call to the end of 'text,
    // which covers every later use of the struct
    fn set_part_from(&'text mut self, new_part: &'text str) {
        self.part = new_part;
    }
}

fn main() {
    let mut excerpt = Excerpt { part: "Call me Ishmael." };
    excerpt.set_part_from("A new excerpt");
    // Error: `excerpt` is still mutably borrowed
    println!("{}", excerpt.part);
}
```

Izlaz

```
error[E0502]: cannot borrow `excerpt.part` as immutable because it is also
 ↪  borrowed as mutable
  --> traits-lifetimes-errors/src/bin/nocompile_set_part_from.rs:17:20
   |
15 |     excerpt.set_part_from("A new excerpt");
   |     ------- mutable borrow occurs here
16 |     // Error: `excerpt` is still mutably borrowed
17 |     println!("{}", excerpt.part);
   |                    ^^^^^^^^^^^^
   |                    |
   |                    immutable borrow occurs here
   |                    mutable borrow later used here

For more information about this error, try `rustc --explain E0502`.
error: could not compile `traits-lifetimes-errors` (bin
 ↪  "nocompile_set_part_from") due to 1 previous error
```

Životni vekovi: `&'text mut self`, objašnjenje

- Struktura je posle poziva zaključana, čak i za čitanje.
- Greška se javlja tek na korišćenju posle poziva: bez poslednjeg `println!` kod bi se kompajlirao.
- Za običnu izmenu polja dovoljno je `&mut self`, pa se `&'text mut self` izbegava.

`'static` i ograničenja

Životni vekovi: `static`

- `'static` životni vek znači da vrednost na koju referenca pokazuje važi do kraja programa. Sama promenljiva sa referencom može da nestane i ranije.
- Svi string literali imaju `'static` životni vek.
- Tekst string literala se čuva direktno u binarnom kodu.
- Kad kompajler predlaže `'static`, uzrok su često neusaglašeni životni vekovi ili „viseće“ reference.

Životni vekovi: primer sa `static`

primeri/traits-lifetimes-errors/src/bin/static.rs

```
fn main() {
    // String literal with a 'static lifetime
    let s1: &'static str = "I have a static lifetime.";
    println!("{}", s1);

    // String allocated at runtime, NOT 'static
    let s2 = String::from("I have a shorter lifetime.");
    // `s2_ref` borrows from `s2`, so it is not 'static
    let s2_ref: &str = &s2;
    println!("{}", s2_ref);

    // Function that accepts a 'static string slice
    print_static_string("This is a static string literal.");
}

// A function that accepts only 'static string slices
fn print_static_string(s: &'static str) {
    println!("Static string: {}", s);
}
```

Izlaz

```
I have a static lifetime.
I have a shorter lifetime.
Static string: This is a static string literal.
```

Životni vekovi: referenca nije `'static` (greška)

primeri/traits-lifetimes-errors/src/bin/nocompile_static.rs

```
fn main() {
    // Error: `s2` is dropped at the end of `main`, so `&s2` is not
    ↪  'static
    let s2 = String::from("I have a shorter lifetime.");
    let s3: &'static str = &s2;
    println!("{}", s3);
```

```
    // Error: the function requires 'static, but `&s4` is not 'static
    let s4 = String::from("Temporary string");
    print_static_string(&s4);
}

// A function that accepts only 'static string slices
fn print_static_string(s: &'static str) {
    println!("Static string: {}", s);
}
```

Izlaz

```
error[E0597]: `s2` does not live long enough
  --> traits-lifetimes-errors/src/bin/nocompile_static.rs:4:28
   |
 3 |     let s2 = String::from("I have a shorter lifetime.");
   |         -- binding `s2` declared here
 4 |     let s3: &'static str = &s2;
   |             ------------   ^^^ borrowed value does not live long
 ↪  enough
   |             |
   |             type annotation requires that `s2` is borrowed for
 ↪  `'static`
...
10 | }
   | - `s2` dropped here while still borrowed

error[E0597]: `s4` does not live long enough
  --> traits-lifetimes-errors/src/bin/nocompile_static.rs:9:25
   |
 8 |     let s4 = String::from("Temporary string");
   |         -- binding `s4` declared here
 9 |     print_static_string(&s4);
   |     --------------------^^^-
   |     |                   |
   |     |                   borrowed value does not live long enough
   |     argument requires that `s4` is borrowed for `'static`
10 | }
   | - `s4` dropped here while still borrowed

For more information about this error, try `rustc --explain E0597`.
error: could not compile `traits-lifetimes-errors` (bin
 ↪  "nocompile_static") due to 2 previous errors
```

Životni vekovi: `static`, objašnjenje

- `s1` je string literal, pa ima životni vek `'static`.
- `s2_ref` pozajmljuje `s2`, pa nije `'static`.
- U grešci `&s2` i `&s4` nisu `'static`, jer se `s2` i `s4` oslobađaju na kraju `main`, pa kompajler prijavljuje `E0597`.

Životni vekovi: generički tipovi i osobine

- Životni vekovi su vrsta generičkih parametara, pa se i u funkcijama i u strukturama navode u istoj listi kao generički tipovi.
- U toj listi životni vekovi moraju da budu ispred tipova: `<'a, T>`.
- Uz njih mogu da stoje i ograničenja osobina (`T: Display`).

Životni vekovi: uz generičke tipove

primeri/traits-lifetimes-errors/src/bin/generics.rs

```
use std::fmt::Display;

fn longest_with_an_announcement<'a, T>(x: &'a str, y: &'a str, ann: T) ->
 ↪  &'a str
where
    T: Display,
{
    println!("Announcement! {ann}");
    if x.len() > y.len() {
        x
    } else {
        y
    }
}

fn main() {
    let string1 = String::from("abcd");
    let string2 = "xyz";

    let result =
        longest_with_an_announcement(string1.as_str(), string2, "Today is
 ↪  someone's birthday!");
    println!("The longest string is {result}");
}
```

Izlaz

```
Announcement! Today is someone's birthday!
The longest string is abcd
```

Životni vekovi: `T: 'static`

- `T: 'static` znači da tip ne sadrži reference kraće od `'static`.
- To ne znači da vrednost živi do kraja programa: vlasnička vrednost (`String`) prolazi i može da se oslobodi ranije.
- Referenca na lokalnu promenljivu ne prolazi: `&local` je tip sa životnim vekom pozajmice, a ona se završava pre kraja programa.
- Ovo ograničenje traži, na primer, `std::thread::spawn`.

Životni vekovi: `T: 'static` (greška)

primeri/traits-lifetimes-errors/src/bin/nocompile_static_bound.rs

```
// `T: 'static`: the type holds no references shorter than 'static
fn keep<T: 'static>(value: T) -> T {
    value
}

fn main() {
    // OK: `String` owns its data and holds no references
    let owned = keep(String::from("owned"));
    println!("{}", owned);

    let local = String::from("local");
    // Error: `&local` is a reference shorter than 'static
```

```
    println!("{}", keep(&local));
}
```

Izlaz

```
error[E0597]: `local` does not live long enough
  --> traits-lifetimes-errors/src/bin/nocompile_static_bound.rs:13:25
   |
11 |     let local = String::from("local");
   |         ----- binding `local` declared here
12 |     // Error: `&local` is a reference shorter than 'static
13 |     println!("{}", keep(&local));
   |                     -----^^^^^^-
   |                     |    |
   |                     |    borrowed value does not live long enough
   |                     argument requires that `local` is borrowed for
 ↪  `'static`
14 | }
   | - `local` dropped here while still borrowed
   |
note: requirement that the value outlives `'static` introduced here
  --> traits-lifetimes-errors/src/bin/nocompile_static_bound.rs:2:12
   |
 2 | fn keep<T: 'static>(value: T) -> T {
   |            ^^^^^^^

For more information about this error, try `rustc --explain E0597`.
error: could not compile `traits-lifetimes-errors` (bin
 ↪  "nocompile_static_bound") due to 1 previous error
```

Životni vekovi: uslov `'new: 'text`

- `'new: 'text` znači da `'new` traje bar koliko `'text`.

Životni vekovi: uslov `'new: 'text`, primer

primeri/traits-lifetimes-errors/src/bin/lifetime_outlives.rs

```
struct Excerpt<'text> {
    part: &'text str,
}

impl<'text> Excerpt<'text> {
    // 'new: 'text means the new text lives at least as long as 'text,
    // the same guarantee as `new_part: &'text str`
    fn set_part<'new>(&mut self, new_part: &'new str)
    where
        'new: 'text,
    {
        self.part = new_part;
    }
}

fn main() {
    let novel = String::from("Call me Ishmael.");
    let mut excerpt = Excerpt { part: &novel };

    // A string literal lives for the whole program
```

```
    excerpt.set_part("A new excerpt");
    println!("{}", excerpt.part);
}
```

Izlaz

```
A new excerpt
```

Životni vekovi: uslov `'new: 'text`, objašnjenje

- Parametar `&'new str` sa tim uslovom prihvata isto što i `&'text str`.
- Dugo živ tekst, na primer string literal, prolazi. Kratko živ ne prolazi: ista greška `E0597` kao kod `&mut self`.
- Bez tog uslova kompajler prijavljuje „lifetime may not live long enough“ i sam predlaže `'new: 'text`.

Rukovanje greškama

Rukovanje greškama

- Rust ne poseduje izuzetke.
- Umesto toga koristi `Result<T, E>` za nadoknadive greške i makro `panic!` za nenadoknadive.
- Greške se dele na:
  - Nadoknadive
    - Otvaranje datoteke koja ne postoji
  - Nenadoknadive
    - Pristup vektoru na indeksu van granica

Nenadoknadive greške

- Makro `panic!` koristi se za fatalne greške.
- Podrazumevano, panika zaustavlja nit u kojoj je nastala. Ako je to glavna nit, završava se ceo program.
- Pritom se radi „unwinding“: Rust prolazi unazad kroz stek i čisti podatke svake funkcije.
- Opcija „abort“ završava program bez čišćenja memorije.
  - Uključuje se sa `panic = 'abort'` u odeljku `[profile]` datoteke `Cargo.toml`.
- Backtrace omogućava praćenje tačke izazivanja panike.
  - Uključuje se promenljivom okruženja `RUST_BACKTRACE=1`.

Nenadoknadive greške, primer

primeri/traits-lifetimes-errors/src/bin/panic.rs

```
fn main() {
    // Example 1: Explicit panic with a custom message
    panic!("Something went terribly wrong!");

    // Example 2: Accessing an invalid index in a vector (implicit panic)
    let numbers = vec![1, 2, 3];
    println!("The number at index 5 is: {}", numbers[5]); // This line
     ↪  will cause a panic!
}
```

Izlaz

```
thread 'main' (4907) panicked at
 ↪  traits-lifetimes-errors/src/bin/panic.rs:3:5:
Something went terribly wrong!
note: run with `RUST_BACKTRACE=1` environment variable to display a
 ↪  backtrace
```

Nadoknadive greške

- Većina grešaka nije razlog za potpuni završetak programa.
- Umesto toga se koristi enum `Result<T, E>`.
- Primer je neuspešno otvaranje datoteke.

Nadoknadive greške, primer

primeri/traits-lifetimes-errors/src/bin/error.rs

```rust
use std::fs::File;

fn main() {
    let greeting_file_result = File::open("hello.txt");

    let greeting_file = match greeting_file_result {
        Ok(file) => file,
        Err(error) => panic!("Problem opening the file: {error:?}"),
    };
}
```

Izlaz

```
thread 'main' (4923) panicked at
 ↪  traits-lifetimes-errors/src/bin/error.rs:8:23:
Problem opening the file: Os { code: 2, kind: NotFound, message: "No such
 ↪  file or directory" }
note: run with `RUST_BACKTRACE=1` environment variable to display a
 ↪  backtrace
```

Greške: poređenje po vrsti (`ErrorKind`)

- Rust omogućava da se odredi koja se greška dogodila.
- `io::Error` je struktura iz standardne biblioteke. Njena metoda `kind` vraća vrednost enuma `io::ErrorKind`, pa je moguće vršiti poređenje po vrsti.

Greške: poređenje po vrsti, primer

primeri/traits-lifetimes-errors/src/bin/error_matching.rs

```rust
use std::fs::File;
use std::io::ErrorKind;

fn main() {
    let greeting_file_result = File::open("hello.txt");

    let greeting_file = match greeting_file_result {
        Ok(file) => {
            println!("Opened hello.txt");
            file
        }
```

```
        Err(error) => match error.kind() {
            ErrorKind::NotFound => match File::create("hello.txt") {
                Ok(fc) => {
                    println!("hello.txt not found, created it");
                    fc
                }
                Err(e) => panic!("Problem creating the file: {e:?}"),
            },
            other_error => {
                panic!("Problem opening the file: {other_error:?}");
            }
        },
    };
}
```

Izlaz

```
hello.txt not found, created it
```

Greške: `unwrap`

- Metoda `unwrap` predstavlja skraćenicu implementiranu nad `Result<T, E>` kao:
  - `match` koji:
    * za `Ok` vraća otpakovanu vrednost
    * za `Err` izaziva paniku

Greške: primer sa `unwrap`

`primeri/traits-lifetimes-errors/src/bin/unwrap_a.rs`

```
use std::fs::File;

// unwrap is a shortcut for a match expression:
// if the Result is Ok, it returns the value inside the Ok;
// if the Result is Err, it calls the panic! macro.
fn main() {
    let greeting_file = File::open("hello.txt").unwrap();
}
```

Izlaz

```
thread 'main' (4955) panicked at
 ↪  traits-lifetimes-errors/src/bin/unwrap_a.rs:7:49:
called `Result::unwrap()` on an `Err` value: Os { code: 2, kind: NotFound,
 ↪  message: "No such file or directory" }
note: run with `RUST_BACKTRACE=1` environment variable to display a
 ↪  backtrace
```

Greške: `unwrap_or` i `unwrap_or_default`

- `unwrap_or(vrednost)` vraća zadatu vrednost umesto panike.
- `unwrap_or_default()` vraća podrazumevanu vrednost tipa (`0` za brojeve, prazan `String`).
- Obe metode postoje i za `Option`.

Greške: `unwrap_or` i `unwrap_or_default`, primer

`primeri/traits-lifetimes-errors/src/bin/unwrap_or.rs`

```rust
fn main() {
    // unwrap_or: a fixed value is used instead of a panic
    let a: i32 = "42".parse().unwrap_or(0);
    let b: i32 = "abc".parse().unwrap_or(0);
    println!("unwrap_or: {a}, {b}");

    // unwrap_or_default: the default value of the type (0 for numbers)
    let c: i32 = "abc".parse().unwrap_or_default();
    println!("unwrap_or_default: {c}");

    // The same methods work on Option
    let name: Option<&str> = None;
    println!("Option: {}", name.unwrap_or("anonymous"));
}
```

Izlaz

```
unwrap_or: 42, 0
unwrap_or_default: 0
Option: anonymous
```

Greške: `unwrap_or_else`

- `unwrap_or_else(f)` poziva zatvorenje `f` samo kad je potrebno, dok se vrednost za `unwrap_or` izračuna uvek, čak i kad je rezultat `Ok`.

primeri/traits-lifetimes-errors/src/bin/unwrap_b.rs

```rust
use std::fs::File;
use std::io::ErrorKind;

fn main() {
    let greeting_file = File::open("hello.txt").unwrap_or_else(|error| {
        if error.kind() == ErrorKind::NotFound {
            File::create("hello.txt").unwrap_or_else(|error| {
                panic!("Problem creating the file: {error:?}");
            })
        } else {
            panic!("Problem opening the file: {error:?}");
        }
    });
    println!("hello.txt is ready");
}
```

Izlaz

```
hello.txt is ready
```

Greške: `expect`

- Metoda koja radi kao `unwrap`, ali u slučaju panike ispisuje prosleđenu poruku.

Greške: `expect`, primer

primeri/traits-lifetimes-errors/src/bin/expect.rs

```rust
use std::fs::File;

// expect is like unwrap, but lets us specify the panic! message
```

```
fn main() {
    let greeting_file =
        File::open("hello.txt").expect("hello.txt should be included in
        ↪  this project");
}
```

Izlaz

```
thread 'main' (5003) panicked at
↪  traits-lifetimes-errors/src/bin/expect.rs:6:33:
hello.txt should be included in this project: Os { code: 2, kind:
↪  NotFound, message: "No such file or directory" }
note: run with `RUST_BACKTRACE=1` environment variable to display a
↪  backtrace
```

Propagacija grešaka

- Ukoliko se greška dogodi prilikom poziva funkcije unutar druge funkcije, moguće je „ne obraditi“ grešku, već proslediti je naviše.
- Ovaj obrazac je čest, pa Rust ima operator ? koji ga skraćuje.
  - Sme da se koristi samo u funkciji čiji povratni tip odgovara vrednosti na koju se primenjuje, na primer `Result` za `Result`.

Greške: propagiranje pomoću match

primeri/traits-lifetimes-errors/src/bin/propagate_a.rs

```
use std::fs::File;
use std::io::{self, Read};

fn read_username_from_file() -> Result<String, io::Error> {
    let username_file_result = File::open("hello.txt");

    let mut username_file = match username_file_result {
        Ok(file) => file,
        Err(e) => return Err(e),
    };

    let mut username = String::new();

    match username_file.read_to_string(&mut username) {
        Ok(_) => Ok(username),
        Err(e) => Err(e),
    }
}

fn main() {
    match read_username_from_file() {
        Ok(username) => println!("Username: {}", username),
        Err(e) => println!("Error: {}", e),
    }
}
```

Izlaz

```
Error: No such file or directory (os error 2)
```

Greške: operator ?

primeri/traits-lifetimes-errors/src/bin/propagate_b.rs

```rust
use std::fs::File;
use std::io::{self, Read};

fn read_username_from_file() -> Result<String, io::Error> {
    let mut username_file = File::open("hello.txt")?;
    let mut username = String::new();
    username_file.read_to_string(&mut username)?;
    Ok(username)
}

fn main() {
    match read_username_from_file() {
        Ok(username) => println!("Username: {}", username),
        Err(e) => println!("Error: {}", e),
    }
}
```

Izlaz

```
Error: No such file or directory (os error 2)
```

Greške: ulančavanje operatora ?

primeri/traits-lifetimes-errors/src/bin/propagate_c.rs

```rust
use std::fs::File;
use std::io::{self, Read};

fn read_username_from_file() -> Result<String, io::Error> {
    let mut username = String::new();
    File::open("hello.txt")?.read_to_string(&mut username)?;
    Ok(username)
}

fn main() {
    match read_username_from_file() {
        Ok(username) => println!("Username: {}", username),
        Err(e) => println!("Error: {}", e),
    }
}
```

Izlaz

```
Error: No such file or directory (os error 2)
```

Greške: `?` u `main` bez `Result` (greška)

primeri/traits-lifetimes-errors/src/bin/nocompile_propagate_d.rs

```rust
use std::fs::File;

fn main() {
    let greeting_file = File::open("hello.txt")?;
}
```

Izlaz

```
error[E0277]: the `?` operator can only be used in a function that returns
 ↪  `Result` or `Option` (or another type that implements `FromResidual`)
 --> traits-lifetimes-errors/src/bin/nocompile_propagate_d.rs:4:48
  |
3 | fn main() {
  | --------- this function should return `Result` or `Option` to accept
 ↪  `?`
4 |     let greeting_file = File::open("hello.txt")?;
  |                                                ^ cannot use the `?`
 ↪  operator in a function that returns `()`
  |
help: consider adding return type
  |
3 ~ fn main() -> Result<(), Box<dyn std::error::Error>> {
4 |     let greeting_file = File::open("hello.txt")?;
5 +     Ok(())
  |

For more information about this error, try `rustc --explain E0277`.
error: could not compile `traits-lifetimes-errors` (bin
 ↪  "nocompile_propagate_d") due to 1 previous error
```

Greške: `main` koji vraća `Result`

- Funkcija `main` može da vrati i `Result<(), E>`.
- Ako vrati `Ok(())`, program se završava sa izlaznim kodom 0.
- Ako vrati `Err`, program se završava sa izlaznim kodom različitim od 0.

Greške: `main` koji vraća `Result`, primer

primeri/traits-lifetimes-errors/src/bin/propagate_main.rs

```rust
use std::error::Error;
use std::fs::File;

fn main() -> Result<(), Box<dyn Error>> {
    let greeting_file = File::open("hello.txt")?;
    println!("Opened: {:?}", greeting_file);
    Ok(())
}
```

Izlaz

```
Error: Os { code: 2, kind: NotFound, message: "No such file or directory"
 ↪  }
```

Greške: `main` koji vraća `Result`, objašnjenje

- `Box<dyn Error>` čitamo kao „bilo koja vrsta greške“.
- `?` vraća grešku iz `main`, a program je ispisuje posle `Error:`.
- `Ok(())` na kraju znači da je `main` uspešno završen.

Greške: sopstveni tip greške

- Funkcija može da vrati jedan tip greške za sve razloge neuspeha.
- Operator `?` poziva funkciju `from` iz osobine `From` i tako pretvara grešku u tip koji funkcija vraća.

Greške: sopstveni tip greške, primer

primeri/traits-lifetimes-errors/src/bin/error_from.rs

```rust
use std::fs;
use std::io;
use std::num::ParseIntError;

// One error type for every way `read_port` can fail
enum ConfigError {
    Io(io::Error),
    Parse(ParseIntError),
}

impl From<io::Error> for ConfigError {
    fn from(e: io::Error) -> Self {
        ConfigError::Io(e)
    }
}

impl From<ParseIntError> for ConfigError {
    fn from(e: ParseIntError) -> Self {
        ConfigError::Parse(e)
    }
}

// `?` calls `from` and converts each error into `ConfigError`
fn read_port(path: &str) -> Result<u16, ConfigError> {
    let text = fs::read_to_string(path)?;
    let port = text.trim().parse::<u16>()?;
    Ok(port)
}

fn report(path: &str) {
    match read_port(path) {
        Ok(port) => println!("{}: port {}", path, port),
        Err(ConfigError::Io(e)) => println!("{}: cannot read: {}", path,
        ↪  e),
        Err(ConfigError::Parse(e)) => println!("{}: not a number: {}",
        ↪  path, e),
    }
}

fn main() {
    report("missing.txt");
    fs::write("port.txt", "abc").unwrap();
    report("port.txt");
    fs::write("port.txt", "8080").unwrap();
    report("port.txt");
    fs::remove_file("port.txt").unwrap();
}
```

Izlaz

```
missing.txt: cannot read: No such file or directory (os error 2)
port.txt: not a number: invalid digit found in string
port.txt: port 8080
```

Greške: sopstveni tip greške, objašnjenje

- `fs::read_to_string` vraća `io::Error`, a `parse` vraća `ParseIntError`.
- `?` ih pretvara u `ConfigError` preko `impl From`, bez dodatnog koda u funkciji `read_port`.
- Bez odgovarajućeg `impl From` kompajler prijavljuje `E0277`.
- `report` po varijanti `ConfigError` zna šta je pošlo naopako.

Greške: `Option` i operator `?`

- Operator `?` može da se koristi i sa vrednostima tipa `Option`.
  - U slučaju `None`
    - Iz cele funkcije se vraća `None`.
  - U slučaju `Some`
    - Vrednost iz `Some` je rezultat izraza i funkcija se nastavlja.
- Nije moguće koristiti:
  - `option?` u okviru funkcije koja vraća `Result`
  - `result?` u okviru funkcije koja vraća `Option`
- U tom slučaju koriste se metode `ok` ili `ok_or`, ili slične.

Option: obrada vrednosti

primeri/traits-lifetimes-errors/src/bin/option.rs

```rust
fn process_number(input: Option<&str>) -> Option<i32> {
    // Step 1: Get the text out of the Option with ? (None returns early)
    let num_str = input?;
    println!("Input text: {:?}", num_str);

    // Step 2: Try parsing the string as an integer with ?
    let num = num_str.parse::<i32>().ok()?;
    println!("Parsed number: {:?}", num);

    // Step 3: Perform a calculation (e.g., double the number)
    let doubled = num * 2;
    println!("Doubled number: {:?}", doubled);
    Some(doubled)
}

fn main() {
    // Some with a valid number: all three steps run
    println!("Processing Some(\"42\"):");
    let input_some = Some("42");
    let result_some = process_number(input_some);
    println!("Result: {:?}", result_some); // Should return Some(84)

    // None: ? returns None after step 1
    println!("Processing None:");
    let input_none: Option<&str> = None;
    let result_none = process_number(input_none);
    println!("Result: {:?}", result_none); // Should return None

    // Some with an invalid number: ? returns None after step 2
    println!("Processing Some(\"not_a_number\"):");
    let input_invalid = Some("not_a_number");
    let result_invalid = process_number(input_invalid);
    println!("Result: {:?}", result_invalid); // Should return None
}
```

Izlaz

```
Processing Some("42"):
Input text: "42"
Parsed number: 42
Doubled number: 84
Result: Some(84)
Processing None:
Result: None
Processing Some("not_a_number"):
Input text: "not_a_number"
Result: None
```

Option i Result: Option u Result

primeri/traits-lifetimes-errors/src/bin/option_to_result.rs

```rust
fn main() {
    // Convert Option to Result using .ok_or()
    let option_to_result: Option<i32> = Some(100);
    let result_from_option = option_to_result.ok_or("No value found"); //
     ↪ Converts Some(100) to Ok(100)

    match result_from_option {
        Ok(value) => println!("Converted Option to Result: Ok({})",
         ↪ value),
        Err(err) => println!("Converted Option to Result: Err({})", err),
    }

    // Example with None
    let none_option: Option<i32> = None;
    let err_from_none = none_option.ok_or("No value present"); // Converts
     ↪ None to Err("No value present")

    match err_from_none {
        Ok(value) => println!("Converted Option to Result: Ok({})",
         ↪ value),
        Err(err) => println!("Converted Option to Result: Err({})", err),
    }
}
```

Izlaz

```
Converted Option to Result: Ok(100)
Converted Option to Result: Err(No value present)
```

Option i Result: Result u Option

primeri/traits-lifetimes-errors/src/bin/result_to_option.rs

```rust
fn main() {
    // Convert Result to Option using .ok()
    let result_to_option: Result<i32, &str> = Ok(42);
    let option_from_result = result_to_option.ok(); // Converts Ok(42) to
     ↪ Some(42)

    match option_from_result {
        Some(value) => println!("Converted Result to Option: Some({})",
         ↪ value),
        None => println!("Converted Result to Option: None"),
```

```
    }

    // Example with Err
    let error_result: Result<i32, &str> = Err("Error occurred");
    let none_from_error = error_result.ok(); // Converts Err to None

    match none_from_error {
        Some(value) => println!("Converted Result to Option: Some({})",
        ↪  value),
        None => println!("Converted Result to Option: None (error was
        ↪  ignored)"),
    }
}
```

Izlaz

```
Converted Result to Option: Some(42)
Converted Result to Option: None (error was ignored)
```

Greške: kada `panic!`, a kada `Result`

- Za funkciju koja može da ne uspe, `Result` je dobar podrazumevani izbor.
  - Posebno kad je neuspeh očekivan, na primer neispravan ulaz parsera.
- `panic!` je prikladan kad program može da dospe u loše stanje (narušena pretpostavka, garancija ili ugovor), a to nije očekivano.
- `unwrap` i `expect` su prikladni:
  - U primerima, prototipovima i testovima.
  - Kad neka logika garantuje `Ok`, a kompajler to ne može da zna. Razlog se navodi u `expect`.

Zadaci

Zadatak 1

Napisati program u Rust programskom jeziku sa osobinom `Licensed` i strukturama `SomeSoftware` (polje `version_number` tipa `i32`) i `OtherSoftware` (polje `version_number` tipa `String`), a zatim:

- Dodati metodi `licensing_info` osobine `Licensed` podrazumevanu implementaciju koja vraća `"Default license"`.
- Implementirati `Licensed` za obe strukture bez sopstvene implementacije metode.
- Napisati funkciju `compare_license_types` koja prima dva argumenta bilo kojih tipova koji implementiraju `Licensed` i vraća `true` ako imaju iste informacije o licenci.
- Pozvati funkciju u `main` i ispisati rezultat.

Zadatak 2

Napisati program u Rust programskom jeziku sa strukturom `Book` koja ima dva polja tipa `&str`, `author` i `title`, a zatim:

- Dodati strukturi `Book` potrebne anotacije životnog veka.
- Napisati funkciju `longest` koja prima dva `&str` i vraća duži od njih.
- Napraviti `Book` u `main` funkciji i pozvati `longest` nad autorom i naslovom.

Zadatak 3

Napisati program u Rust programskom jeziku sa:

- Funkcijom `total_cost` koja prima količinu predmeta kao `&str` i vraća `Result<i32, ParseIntError>`. Cena jednog predmeta je 5, a naknada za obradu 1. Ako se količina ne može pretvoriti u broj, funkcija odmah vraća grešku (operator `?`).
- Strukturom `PositiveNonzeroInteger(u64)` i enumeracijom `CreationError` sa varijantama `Negative` i `Zero`. Funkcija `PositiveNonzeroInteger::new(i64)` vraća `Result<Self, CreationError>`: grešku za negativan broj ili nulu, a inače `Ok`.
- Pozivima u `main` funkciji za ispravan i neispravan unos.

Izvori

Izvori

- Rust Community. “The Rust Programming Language - the Rust Programming Language.” Rust-Lang.org, 2018, doc.rust-lang.org/book/.
- Crichton, Will. “Experiment Introduction - the Rust Programming Language.” Brown.edu, rust-book.cs.brown.edu/.
- Rust Community. “Tour of Rust - Let’s Go on an Adventure!” Tourofrust.com, tourofrust.com/.
- Rust Team. “Rust Programming Language.” Rust-Lang.org, 2018, www.rust-lang.org/.
- Rust Community. “Rustlings.” GitHub, github.com/rust-lang/rustlings (vežbe `15_traits`: `traits3.rs`, `traits4.rs`; `16_lifetimes`: `lifetimes1.rs`, `lifetimes3.rs`; `13_error_handling`: `errors2.rs`, `errors4.rs`).
- Stack Overflow. “When is it useful to define multiple lifetimes in a struct?” stackoverflow.com/questions/29861388/when-is-it-useful-to-define-multiple-lifetimes-in-a-struct (primer `lifetime_two_fields.rs`).

Izvori: pojedinačne tvrdnje

- Rust Community. “2094-nll.” The Rust RFC Book, rust-lang.github.io/rfcs/2094-nll.html (živost životnih vekova, ograničenja i aktivne pozajmice).
- Rust Community. “Static.” Rust By Example, doc.rust-lang.org/rust-by-example/scope/lifetime/static_lifetime.html (`'static` kao životni vek reference i kao ograničenje tipa).
- Rust Community. “Macro panic.” The Rust Standard Library, doc.rust-lang.org/std/macro.panic.html (panika zaustavlja nit).
- quinedot. “Borrowing rules about self.” The Rust Programming Language Forum, users.rust-lang.org/t/borrowing-rules-about-self/69451/2 (`&'a mut self` kao anti-obrazac).
- pretzelhammer. “Common Rust Lifetime Misconceptions.” GitHub, github.com/pretzelhammer/rust-blog/blob/master/posts/common-rust-lifetime-misconceptions.md (zabluda 5: metoda sa `&'a mut self`).